

Bambu Printer Status JSON Setup Instructions

These instructions explain how to create and maintain a `status.json` file for the Tidbyt Bambu Printer Status app using a small Python script. The updated version below uses generic home-directory placeholders instead of personal account paths, so you should replace `your_username` with your own Linux username or use `~` for your home directory.[\[web:269\]](#)[\[web:272\]](#)

Path placeholders

Use either of these styles throughout the setup:

- `/home/your_username/bambu-cloud/`
- `~/bambu-cloud/`

For example:

- Script file: `~/bambu-cloud/cloud_status_update.py`
- Output JSON: `~/bambu-cloud/status.json`
- Token file: `~/.bambu_token`

The `~` character expands to the current user's home directory in common Unix shells, which makes it a safer and more portable documentation pattern than hardcoding a specific username.[\[web:272\]](#)

Requirements

Install Python 3 and the `requests` package on the machine that will generate the JSON file.

```
python3 --version
python3 -m pip install --user requests
```

Token file

Create a token file in your home directory named `~.bambu_token`.

```
cat > ~/.bambu_token <<'EOF_TOKEN'
YOUR_BAMBU_TOKEN_HERE
EOF_TOKEN
chmod 600 ~/.bambu_token
```

If your token file contains JSON instead of a plain token string, the script also accepts either a `token` or `access_token` property.

Create the working folder

```
mkdir -p ~/bambu-cloud
```

Write the update script

This version is updated to preserve historical `last\_completed\_\*` values when the upstream task API returns partial data, instead of overwriting good values with missing ones.[web:256][web:260]

```
cat <<'PYEOF' > ~/bambu-cloud/cloud_status_update.py
#!/usr/bin/env python3

import json
from datetime import datetime, timezone
from pathlib import Path

import requests

STATUS_PATH = Path.home() / "bambu-cloud" / "status.json"
TOKEN_PATH = Path.home() / ".bambu_token"
API_BASE = "https://api.bambulab.com"

def iso_now():
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat()

def read_token():
    raw = TOKEN_PATH.read_text().strip()
    if raw.startswith("{}"):
        try:
            data = json.loads(raw)
            t = data.get("token") or data.get("access_token")
            if t:
                return t.strip()
        except Exception:
            pass
    return raw

def api_get(path, token, params=None):
    url = f"{API_BASE}{path}"
    headers = {
        "Authorization": f"Bearer {token}",
        "Accept": "application/json",
    }
    resp = requests.get(url, headers=headers, params=params, timeout=15)
    resp.raise_for_status()
    return resp.json()

def fmt_duration(seconds):
    if seconds is None:
        return None
    try:
        seconds = int(seconds)
    except Exception:
        return None

    if seconds < 60:
        return f"{seconds}s"
    minutes, _ = divmod(seconds, 60)
    if minutes < 60:
        return f"{minutes}m"
    hours, minutes = divmod(minutes, 60)
    if minutes == 0:
        return f"{hours}h"
    return f"{hours}h {minutes}m"

def normalize_state(online, raw_status, cloud_print_job):
    if not online:
        return "OFFLINE"

    status = (raw_status or "").upper()
    if status in ("RUNNING", "PRINTING"):
        return "PRINTING"
    if status in ("PAUSE", "PAUSED"):
        return "PAUSED"
    if status in ("SUCCESS", "FINISH", "COMPLETED", "IDLE", ""):
        return "IDLE"
    if cloud_print_job:
        return "IDLE"
    return status

def list_printers(token):
```

```

data = api_get("/v1/iot-service/api/user/bind", token)
return data.get("devices") or data.get("data") or []

def fetch_tasks(token):
    data = api_get("/v1/user-service/my/tasks", token)
    hits = data.get("hits")
    return hits if isinstance(hits, list) else []

def index_tasks_by_device_and_id(tasks):
    tasks_by_id = {}
    tasks_by_device = {}

    for t in tasks:
        if not isinstance(t, dict):
            continue

        device_id = t.get("deviceId")
        job_id = t.get("id")
        if not device_id:
            continue

        if job_id is not None:
            try:
                tasks_by_id[(str(device_id), int(job_id))] = t
            except Exception:
                pass

        tasks_by_device.setdefault(str(device_id), []).append(t)

    for dev_id, lst in tasks_by_device.items():
        lst.sort(key=lambda task: task.get("startTime") or "", reverse=True)

    return tasks_by_id, tasks_by_device

def pick_task_for_printer(device_id, cloud_print_job, tasks_by_id, tasks_by_device):
    dev_key = str(device_id)

    if cloud_print_job is not None:
        try:
            hit = tasks_by_id.get((dev_key, int(cloud_print_job)))
            if hit:
                return hit
        except Exception:
            pass

    dev_tasks = tasks_by_device.get(dev_key) or []
    if dev_tasks:
        return dev_tasks[0]

    return None

def extract_job_fields_from_task(task):
    if not isinstance(task, dict):
        return None, None, None, None, None

    job_name = task.get("title") or task.get("designTitle") or None
    job_objects = task.get("designTitle") or None
    estimated_seconds = task.get("costTime")
    estimated_text = fmt_duration(estimated_seconds)
    end_time = task.get("endTime") or None

    return job_name, job_objects, estimated_seconds, estimated_text, end_time

def load_previous_status():
    if not STATUS_PATH.exists():
        return {}

    try:
        data = json.loads(STATUS_PATH.read_text())
        printers = data.get("printers", [])
        out = {}
        for p in printers:
            pid = p.get("id")
            if pid:
                out[str(pid)] = p
    
```

```

        return out
    except Exception:
        return {}

def build_printer_record(device, task, previous, now_iso):
    printer_id = str(device.get("dev_id") or device.get("id") or device.get("sn") or "")
    printer_name = device.get("name") or device.get("dev_name") or device.get("device_name") or printer_id
    online = bool(device.get("online", False))
    raw_status = device.get("print_status") or device.get("status") or device.get("stg_cur") or ""
    cloud_print_job = device.get("print_job")

    state = normalize_state(online, raw_status, cloud_print_job)

    job_name, job_objects, est_seconds, est_text, task_end_time = extract_job_fields_from_task(task)

    progress = None
    time_left = None

    if state == "PRINTING" and task is not None:
        start_str = task.get("startTime")
        if start_str:
            try:
                ts = start_str.replace("Z", "+00:00")
                start_dt = datetime.fromisoformat(ts)
                now_dt = datetime.now(timezone.utc)
                elapsed = max((now_dt - start_dt).total_seconds(), 0)
                if est_seconds:
                    total = max(int(est_seconds), 1)
                    progress = min(100, max(0, int(elapsed * 100 // total)))
                    time_left = max(total - int(elapsed), 0)
            except Exception:
                pass

    prev_state = (previous or {}).get("state")
    prev_job = (previous or {}).get("cloud_print_job")
    prev_idle_since = (previous or {}).get("idle_since")

    prev_last_completed_job_id = (previous or {}).get("last_completed_job_id")
    prev_last_completed_job_name = (previous or {}).get("last_completed_job_name")
    prev_last_completed_job_objects = (previous or {}).get("last_completed_job_objects")
    prev_last_completed_end_time = (previous or {}).get("last_completed_end_time")
    prev_last_completed_duration_seconds = (previous or {}).get("last_completed_duration_seconds")
    prev_last_completed_duration_text = (previous or {}).get("last_completed_duration_text")

    idle_since = None

    last_completed_job_id = prev_last_completed_job_id
    last_completed_job_name = prev_last_completed_job_name
    last_completed_job_objects = prev_last_completed_job_objects
    last_completed_end_time = prev_last_completed_end_time
    last_completed_duration_seconds = prev_last_completed_duration_seconds
    last_completed_duration_text = prev_last_completed_duration_text

    if state == "PRINTING":
        idle_since = None
    else:
        if prev_state == "PRINTING":
            idle_since = now_iso

            if cloud_print_job is not None:
                last_completed_job_id = cloud_print_job
            elif prev_job is not None:
                last_completed_job_id = prev_job

            if job_name:
                last_completed_job_name = job_name

            if job_objects:
                last_completed_job_objects = job_objects

            if task_end_time:
                last_completed_end_time = task_end_time
            elif not last_completed_end_time:
                last_completed_end_time = now_iso

            if est_seconds is not None:
                last_completed_duration_seconds = est_seconds
                last_completed_duration_text = est_text
            elif prev_state in ("IDLE", "PAUSED", "OFFLINE") and prev_idle_since not in (None, ""):

```

```

        idle_since = prev_idle_since
    else:
        idle_since = None

    if last_completed_job_id is None and cloud_print_job is not None:
        last_completed_job_id = cloud_print_job

    if last_completed_job_name is None and job_name is not None:
        last_completed_job_name = job_name

    if last_completed_job_objects is None and job_objects is not None:
        last_completed_job_objects = job_objects

    if last_completed_end_time is None and task_end_time is not None:
        last_completed_end_time = task_end_time

    if last_completed_duration_seconds is None and est_seconds is not None:
        last_completed_duration_seconds = est_seconds
        last_completed_duration_text = est_text

    if state == "PRINTING":
        display_job_name = job_name
        display_job_objects = job_objects
        display_est_seconds = est_seconds
        display_est_text = est_text
    else:
        display_job_name = last_completed_job_name or job_name
        display_job_objects = last_completed_job_objects or job_objects
        display_est_seconds = last_completed_duration_seconds if last_completed_duration_seconds is not None else 0
        display_est_text = last_completed_duration_text or est_text

    return {
        "id": printer_id,
        "name": printer_name,
        "online": online,
        "state": state,
        "label": raw_status,
        "cloud_print_status": raw_status,
        "cloud_print_job": cloud_print_job,
        "job_name": display_job_name,
        "job_objects": display_job_objects,
        "progress": progress,
        "time_left": time_left,
        "estimated_duration_seconds": display_est_seconds,
        "estimated_duration_text": display_est_text,
        "idle_since": idle_since,
        "last_completed_job_id": last_completed_job_id,
        "last_completed_job_name": last_completed_job_name,
        "last_completed_job_objects": last_completed_job_objects,
        "last_completed_end_time": last_completed_end_time,
        "last_completed_duration_seconds": last_completed_duration_seconds,
        "last_completed_duration_text": last_completed_duration_text,
        "probe_errors": [],
    }

def main():
    token = read_token()
    printers_out = []
    top_errors = []
    now_iso = iso_now()
    previous_by_id = load_previous_status()

    try:
        devices = list_printers(token)
    except Exception as e:
        devices = []
        top_errors.append(f"list_printers: {repr(e)}")

    try:
        tasks = fetch_tasks(token)
        tasks_by_id, tasks_by_device = index_tasks_by_device_and_id(tasks)
    except Exception as e:
        tasks_by_id, tasks_by_device = {}, {}
        top_errors.append(f"fetch_tasks: {repr(e)}")

    for device in devices:
        try:
            printer_id = str(device.get("dev_id") or device.get("id") or device.get("sn") or "")
            cloud_print_job = device.get("print_job")

```

```

        task = pick_task_for_printer(printer_id, cloud_print_job, tasks_by_id, tasks_by_device)
        previous = previous_by_id.get(printer_id, {})
        printers_out.append(build_printer_record(device, task, previous, now_iso))
except Exception as e:
    printers_out.append({
        "id": str(device.get("dev_id") or "unknown"),
        "name": device.get("name") or "Unknown Printer",
        "online": False,
        "state": "ERROR",
        "label": "ERROR",
        "cloud_print_status": None,
        "cloud_print_job": None,
        "job_name": None,
        "job_objects": None,
        "progress": None,
        "time_left": None,
        "estimated_duration_seconds": None,
        "estimated_duration_text": None,
        "idle_since": None,
        "last_completed_job_id": None,
        "last_completed_job_name": None,
        "last_completed_job_objects": None,
        "last_completed_end_time": None,
        "last_completed_duration_seconds": None,
        "last_completed_duration_text": None,
        "probe_errors": [f"build_printer_record: {repr(e)}"],
    })

payload = {
    "updated_at": now_iso,
    "printers": printers_out,
    "probe_errors": top_errors,
}

STATUS_PATH.write_text(json.dumps(payload, indent=2))
print(f"Wrote {STATUS_PATH} with {len(printers_out)} printers")

if __name__ == "__main__":
    main()
PYEOF
chmod +x ~/bambu-cloud/cloud_status_update.py
python3 -m py_compile ~/bambu-cloud/cloud_status_update.py

```

Run the script manually

```

~/bambu-cloud/cloud_status_update.py
cat ~/bambu-cloud/status.json

```

Cron example

Run the script every minute to keep `status.json` fresh.

```
crontab -e
```

Add this line:

```
* * * * * /usr/bin/env python3 /home/your_username/bambu-cloud/cloud_status_update.py >/tmp/bambu-cloud.log 2>&
```

If you prefer, you can also write the cron line using `~`, but the explicit `/home/your\_username/...` pattern is often clearer in documentation and easier to validate when troubleshooting. Linux home directories are commonly under `/home/username/`, but the exact username and path must match the local account on that machine.[web:269]

Tidbyt app configuration

Point the Tidbyt app to the generated `status.json` URL, for example:

- `http://your-server:8787/status.json`
- `http://raspberrypi.local:8787/status.json`

Your web server must expose the generated file over HTTP so the Tidbyt app can retrieve it.

Notes

- Replace `your_username` with your own Linux account name, or use `~` where appropriate.[web:272]
- Keep `.bambu_token` readable only by your user.
- The updated script is more defensive about missing API fields and preserves previous `last_completed_*` values when the cloud API returns incomplete task data.[web:256][web:260]